

K.R. MANGALAM UNIVERSITY

SCHOOL OF ENGINEERING AND TECHNOLOGY



OPERATING SYSTEMS LAB FILE

COURSE CODE: ENCS351

Submitted by

Name – Ishika

Roll no. – 2301010242

Program – BTech CSE

Section – D

Submitted to

Mr. Satinder Pal Singh

Serial No.	Assignments	Page No.
1.	Assignment 1 1.1 Process Creation Utility. 1.2 Command Execution Using exec(). 1.3 Zombie & Orphan Processes. 1.4 Inspecting Process Info from /proc. 1.5 Task 5: Process Prioritization.	3-16
2.	Assignment 2 2.1 Write a Python script to simulate a basic system startup sequence. 2.2 Use the multiprocessing module to create at least two child processes. 2.3 Implement proper logging for process events. 2.4 Generate a log file (process_log.txt). 2.5 Submit script & short report.	17-19
3.	Assignment 3 3.1 CPU Scheduling with Gantt Chart. 3.2 Sequential File Allocation. 3.3 Indexed File Allocation. 3.4 Contiguous Memory Allocation. 3.5 MFT & MVT Memory Management.	20-34
4.	Assignment 4 4.1 Batch Processing Simulation (Python). 4.2 System Startup and Logging. 4.3 System Calls and IPC (fork, exec, pipe). 4.4 VM Detection and Shell Interaction. 4.5 CPU Scheduling Algorithms.	35-51

LAB ASSIGNMENT 1

Task 1: Process Creation Utility

Write a Python program that creates N child processes using `os.fork()`. Each child prints:

- Its PID
- Its Parent PID
- A custom message

The parent should wait for all children using `os.wait()`.

```
import os
import time
import multiprocessing

def child_task(index):
    print(f"[Child {index+1}]")
    print(f"PID : {os.getpid()}")
    print(f"Parent PID : {os.getppid()}")
    print(f"Message : Hello! I am Child {index+1}\n")
    time.sleep(1)

def task1_create_processes(n=3):
    print(f"[Parent] PID: {os.getpid()} - Creating {n} child processes...\n")

    processes = []
    for i in range(n):
        p = multiprocessing.Process(target=child_task,
                                   args=(i,))
        p.start()
        processes.append(p)

    for p in processes:
        p.join()
        print(f"[Parent] Child with PID {p.pid} has terminated.")

if __name__ == "__main__":
    task1_create_processes(n=3)
```

Result:

```
Traceback (most recent call last):
  File "d:\coding_classes\somocode.py", line 27, in <module>
    task1_create_processes(n=3)
  File "d:\coding_classes\somocode.py", line 8, in task1_create_processes
    pid = os.fork()
          ^^^^^^^
AttributeError: module 'os' has no attribute 'fork'

D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somocode.py
[Parent] PID: 34788 - Creating 3 child processes...

[Child 1]
  PID      : 3488
  Parent PID : 34788
  Message   : Hello! I am Child 1

[Child 2]
  PID      : 37276
  Parent PID : 34788
  Message   : Hello! I am Child 2

[Child 3]
  PID      : 20352
  Parent PID : 34788
  Message   : Hello! I am Child 3

[Parent] Child with PID 3488 has terminated.
[Parent] Child with PID 37276 has terminated.
[Parent] Child with PID 20352 has terminated.

D:\coding_classes>
```

Task 2: Command Execution Using exec()

Modify Task 1 so that each child process executes a Linux command (ls, date, ps, etc.) using os.execvp() or subprocess.run().

```
import os
import subprocess

def task2_exec_commands(commands):
    print(f"[Parent] PID: {os.getpid()} - Executing {len(commands)} commands...\n")

    for i, cmd in enumerate(commands):
        print(f"[Child {i+1}] Executing: {' '.join(cmd)}")
        try:
            result = subprocess.run(cmd, capture_output=True, text=True,
                                   check=True, shell=True)
            print(f"[Child {i+1}] Output:\n{result.stdout}")
        except subprocess.CalledProcessError as e:
            print(f"[Child {i+1}] Failed to execute command: {e}")
        print(f"Error Output:\n{e.stderr}")

    print("\n[Parent] All commands executed.")

if __name__ == "__main__":
    commands = [
        ["cmd", "/c", "dir"],      # replaces 'ls -l'
        ["cmd", "/c", "date /T"],  # prints current date
        ["cmd", "/c", "tasklist"]  # replaces 'ps aux'
    ]
    task2_exec_commands(commands)
```

Result:

Output:

[Parent] PID: 9420 - Spawning 3 child processes to execute commands...

[Child 1] Executing: ls -l

[Parent] PID: 9420 - Spawning 3 child processes to execute commands...

[Child 3] Executing: ps aux

[Parent] PID: 9420 - Spawning 3 child processes to execute commands...

[Child 2] Executing: date

total 8

-rw-r--r-- 1 root root 991 Sep 23 04:56 main.py

-rw-r--r-- 1 root root 773 Sep 23 04:56 script1.py

Tue Sep 23 04:56:19 AM UTC 2025

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	1.3	1176384	53544	?	Ssl	02:19	0:01	npm start
root	17	0.1	1.7	1315376	69628	?	Sl	02:19	0:16	node index.js
coderun+	9419	0.0	0.0	5692	1072	?	S	04:56	0:00	timeout 7 python3 main.py
coderun+	9420	50.0	0.2	17804	9752	?	S	04:56	0:00	python3 main.py
coderun+	9422	0.0	0.0	0	0	?	Z	04:56	0:00	[date] <defunct>
coderun+	9423	0.0	0.1	10880	4228	?	R	04:56	0:00	ps aux

[Parent] PID: 9420 - Spawning 3 child processes to execute commands...

[Parent] Child with PID 9421 has terminated.

[Parent] Child with PID 9422 has terminated.

[Parent] Child with PID 9423 has terminated.

Task 3: Zombie & Orphan Processes

Zombie: Fork a child and skip wait() in the parent.

Orphan: Parent exits before the child finishes.

Use `ps -el | grep defunct` to identify zombies.

```
import os
import time

def create_zombie():
    pid = os.fork()

    if pid == 0:
        print(f"Child (PID {os.getpid()}) exiting")
        os._exit(0)
    else:
        print(f"Parent (PID {os.getpid()}) sleeping, child should be zombie")

        time.sleep(30)
        print("Parent exiting and cleaning up child")
        os.wait()

if __name__ == "__main__":
    create_zombie()
```

Result:

```
D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somecode.py
[Parent] PID: 30472 - Creating a child process...

[Parent] Child PID: 24104
[Parent] Simulating zombie (child exited, but parent hasn't joined yet)...

[Child] PID: 24104 - Exiting immediately.
[Parent] Cleaning up the zombie by joining child process.
[Parent] Child 24104 cleaned up properly.

D:\coding_classes>
```

```

import multiprocessing
import os
import time

def child_process():
    print(f'[Child] PID: {os.getpid()} - Parent may exit soon.')
    time.sleep(5)
    print(f'[Child] PID: {os.getpid()} - Still running after
parent exited.\n")

def create_orphan_simulation():
    print(f'[Parent] PID:
{os.getpid()} - Creating a child process...\n")

    p = multiprocessing.Process(target=child_process)
    p.start()

    print(f'[Parent] Exiting immediately. Child ({p.pid}) becomes orphan-like.")
    # Parent process ends here without joining the child
# This simulates an orphan process on Windows
    return

if __name__ == "__main__":
    create_orphan_simulation()

```

Result:

```

D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somecode.py
[Parent] PID: 1436 - Creating a child process...

[Parent] Exiting immediately. Child (33076) becomes orphan-like.
[Child] PID: 33076 - Parent may exit soon.
[Child] PID: 33076 - Still running after parent exited.

D:\coding_classes>

```


Task 4: Inspecting Process Info from /proc

Take a PID as input. Read and print.

- Process name, state, memory usage from /proc/[pid]/status
- Executable path from /proc/[pid]/exe
- Open file descriptors from /proc/[pid]/fd

```
import psutil
import os
import sys

def print_proc_status(proc):
    print("\n Process Status Info:")
    print(f" Name      : {proc.name()}")
    print(f" PID       : {proc.pid}")
    print(f" Status    : {proc.status()}")
    print(f" Memory (MB) : {proc.memory_info().rss / 1024 / 1024:.2f}")
    print(f" CPU Percent : {proc.cpu_percent(interval=0.1)}%")

def print_exe_path(proc):
    print("\n Executable Path:")
    try:
        print(f" {proc.exe()}")
    except Exception as e:
        print(f" Failed to get executable path: {e}")

def print_open_files(proc):
    print("\n Open Files:")
```

```

try:
    files =
proc.open_files()    if
files:                for f in files:
    print(f" - {f.path}")
else:
    print(" (No open files found)")
except Exception as e:
    print(f" Failed to list open files: {e}")

def main():    if
len(sys.argv) > 1:
try:
    pid = int(sys.argv[1])
proc = psutil.Process(pid)
except Exception as e:
    print(f" Invalid PID: {e}")    return    else:    proc
= psutil.Process(os.getpid()) # default to current process

    print_proc_status(proc)
print_exe_path(proc)
print_open_files(proc)

if __name__ == "__main__":
main()

```

Result:

```

D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somocode.py
[Parent] PID: 1436 - Creating a child process...

[Parent] Exiting immediately. Child (33076) becomes orphan-like.
[Child] PID: 33076 - Parent may exit soon.
D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somocode.py
[Parent] PID: 1436 - Creating a child process...

[Parent] Exiting immediately. Child (33076) becomes orphan-like.
D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somocode.py
[Parent] PID: 1436 - Creating a child process...

D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somocode.py
[Parent] PID: 1436 - Creating a child process...
D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somocode.py
D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somocode.py
[Parent] PID: 1436 - Creating a child process...

[Parent] Exiting immediately. Child (33076) becomes orphan-like.
[Child] PID: 33076 - Parent may exit soon.
[Child] PID: 33076 - Still running after parent exited.

D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somocode.py
Usage: python3 d:/coding_classes/somocode.py <PID>

D:\coding_classes>

```

Task 5: Process Prioritization

Create multiple CPU-intensive child processes. Assign different nice() values. Observe and log execution order to show scheduler impact.

```
import
multiprocessing
import os
import time
import psutil

def cpu_intensive_task(nice_val, duration=5):
    p =
    psutil.Process(os.getpid())
    try:
        p.nice(nice_val)

    except Exception as e:
        print(f"[PID {p.pid}] Failed to set nice({nice_val}): {e}")

    print(f"[PID {p.pid}] started with nice
    {p.nice()}")    end_time = time.time() +
    duration    count = 0    while time.time() <
    end_time:
        count += 1    print(f"[PID {p.pid}] finished
    after counting {count}")    return count

def main():
    nice_values = [0, 5, 10, 15, 19]
    duration = 5
    processes = []

    print(f"[Parent PID {os.getpid()}] Spawning processes...\n")

    for nv in nice_values:
        p = multiprocessing.Process(target=cpu_intensive_task, args=(nv,
    duration))    p.start()    processes.append(p)

    for p in
    processes:
        p.join()
```

```
print("\nAll processes have
finished.") if __name__ ==
"__main__": main()
```

Results:

```
D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somecode.py
Usage: python3 d:/coding_classes/somecode.py <PID>
```

```
D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somecode.py
Parent PID 37168 spawning children with different nice values...
```

```
Traceback (most recent call last):
  File "d:\coding_classes\somecode.py", line 38, in <module>
    main()
  File "d:\coding_classes\somecode.py", line 21, in main
    pid = os.fork()
          ^^^^^^^
AttributeError: module 'os' has no attribute 'fork'
```

AttributeError: module 'os' has no attribute 'fork'

```
D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somecode.py
[Parent PID 35228] Spawning processes...
```

```
[PID 35628] Failed to set nice(5): [WinError 87] The parameter is incorrect
[PID 18440] Failed to set nice(0): [WinError 87] The parameter is incorrect
[PID 35628] started with nice 32
[PID 18440] started with nice 32
[PID 31468] Failed to set nice(10): [WinError 87] The parameter is incorrect
[PID 31468] started with nice 32
[PID 36512] Failed to set nice(15): [WinError 87] The parameter is incorrect
[PID 36512] started with nice 32
[PID 27008] Failed to set nice(19): [WinError 87] The parameter is incorrect
[PID 27008] started with nice 32
[PID 35628] finished after counting 41001852
[PID 18440] finished after counting 41448730
[PID 31468] finished after counting 41863806
[PID 36512] finished after counting 41835882
[PID 27008] finished after counting 40982718
```

All processes have finished.

```
D:\coding_classes>
```

LAB ASSIGNMENT 2

Problem Title:

Implementation of System Startup, Process Creation, and Termination Simulation in Python.

Stepwise Subtask Allocation:

Sub-Task 1: Initialize the logging configuration

Objective: Set up the logging system to log messages with timestamps and process names.

```
import logging # Setup logger logging.basicConfig(
    filename='process_log.txt',      level=logging.INFO,
    format='%(asctime)s - %(processName)s - %(message)s'
)
```

Sub-Task 2: Define a function that simulates a process task

Objective: Write a function that mimics the work of a system process.

```
import time
# Dummy function to simulate a task
def system_process(task_name):
    logging.info(f'{task_name} started')
    time.sleep(2) # Simulate task delay
    logging.info(f'{task_name} ended')
```

Sub-Task 3: Create at least two processes and start them concurrently

Objective: Use the multiprocessing module to initiate parallel tasks.

```
import multiprocessing if
__name__ == '__main__':
    print("System
Starting...") # Create processes
    p1 = multiprocessing.Process(target=system_process, args=('Process-1',))
    p2 = multiprocessing.Process(target=system_process, args=('Process-2',))
    # Start processes
    p1.start()
    p2.start()
```

Sub-Task 4: Ensure proper termination and verify logs

Objective: Wait for processes to complete and confirm the shutdown.

```
# Wait for processes to complete
    p1.join()
    p2.join()
    print("System Shutdown.")
```

Program Code with Explanation:

The following Python script simulates a simple system startup. It creates a few dummy processes which perform simple tasks. Each process logs its start and end, and the logger records the process flow, simulating system behavior.

```

# Import required libraries
import multiprocessing
import time
import logging

# Setup logger logging.basicConfig(filename='process_log.txt',
level=logging.INFO,          format='%(asctime)s -
%(processName)s - %(message)s')

# Dummy function to simulate a task
def system_process(task_name):
    logging.info(f'{task_name} started')
    time.sleep(2)
    logging.info(f'{task_name} ended')

if __name__ == '__main__':
    print("System Starting...")

    # Create processes p1 = multiprocessing.Process(target=system_process,
args=('Process-1',)) p2 =
multiprocessing.Process(target=system_process, args=('Process-2',))

    # Start processes
    p1.start()
    p2.start()

    # Wait for processes to complete
    p1.join() p2.join()

    print("System Shutdown.")

```

Results:

```
somecode.py > ...
1  # Import required libraries
2  import multiprocessing
3  import time
4  import logging
5
6  # Setup logger
7  logging.basicConfig(filename='process_log.txt', level=logging.INFO,
8  | | | | | format='%(asctime)s - %(processName)s - %(message)s')
9
10 # Dummy function to simulate a task
11 def system_process(task_name):
12     logging.info(f"{task_name} started")
13     time.sleep(2)
14     logging.info(f"{task_name} ended")
15
16 if __name__ == '__main__':
17     print("System Starting...")
18
19     # Create processes
20     p1 = multiprocessing.Process(target=system_process, args=('Process-1',))
21     p2 = multiprocessing.Process(target=system_process, args=('Process-2',))
22
23     # Start processes
24     p1.start()
25     p2.start()
26
27     # Wait for processes to complete
28     p1.join()
29     p2.join()
30
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER

```
D:\coding_classes>"C:/Program Files/Python312/python.exe" d:/coding_classes/somecode.py
System Starting...
System Shutdown.

D:\coding_classes>
```

Expected Output:

- The terminal displays 'System Starting...' and 'System Shutdown.'
- A log file 'process_log.txt' is generated which records the start and end of each process.

LAB ASSIGNMENT 3

Problem Title:

Simulation of File Allocation, Memory Management, and Scheduling in Python.

Assignment Tasks:

Task 1: CPU Scheduling with Gantt Chart

```
processes = []

n = int(input("Enter number of processes: "))

# Input burst time and priority
for i in range(n):
    bt = int(input(f"Enter Burst Time for P{i+1}: "))    pr = int(input(f"Enter
Priority (lower number = higher priority) for P{i+1}: "))
    processes.append([i+1, bt, pr])

# Sort by priority (ascending)
processes.sort(key=lambda x: x[2])

wt = 0 total_wt
= 0 total_tat =
0

print("\nPriority Scheduling:")
print("PID\tBT\tPriority\tWT\tTAT")

for pid, bt, pr in processes:
    tat = wt + bt # turnaround time
    print(f"{pid}\t{bt}\t{pr}\t\t{wt}\t{tat}")
    total_wt += wt    total_tat += tat    wt += bt #
    update waiting time for next process

print(f"\nAverage Waiting Time: {total_wt / n:.2f}") print(f"Average
Turnaround Time: {total_tat / n:.2f}")
```

Results:

```
Active code page: 65001

D:\Lab-sheet-3>"C:/Program Files/Python312/python.exe" d:/Lab-sheet-3/task1.py
Enter number of processes: 2
Enter Burst Time for P1: 45
Enter Priority (lower number = higher priority) for P1: 3
Enter Burst Time for P2: 37
Enter Priority (lower number = higher priority) for P2: 2

Priority Scheduling:
PID    BT    Priority    WT    TAT
2      37     2          0     37
1      45     3         37    82

Average Waiting Time: 18.50
Average Turnaround Time: 59.50

D:\Lab-sheet-3>
```

Task 2: Sequential File Allocation

Objective: Write a Python program to simulate sequential file allocation strategy.

```
total_blocks = int(input("Enter total number of blocks: "))
block_status = [0] * total_blocks

n = int(input("Enter number of files: "))

for i in range(n):
    start = int(input(f"Enter starting block for file {i+1}: "))
    length = int(input(f"Enter length of file {i+1}: "))

    allocated = True

    for j in range(start, start + length):
        if j
        >= total_blocks or block_status[j] == 1:
            allocated = False
            break

    if allocated:
        for j in range(start,
start + length):
            block_status[j] = 1
            print(f"File {i+1} allocated from block
{start} to {start + length - 1}")
    else:
        print(f"File {i+1} cannot be allocated.")
```

Results:

```
D:\Lab-sheet-3>"C:/Program Files/Python312/python.exe" d:/Lab-sheet-3/task2.py
Enter total number of blocks: 3
Enter number of files: 3
Enter starting block for file 1: 2
Enter length of file 1: 3
File 1 cannot be allocated.
Enter starting block for file 2: 1
Enter length of file 2: 4
File 2 cannot be allocated.
Enter starting block for file 3: 0
Enter length of file 3: 3
File 3 allocated from block 0 to 2

D:\Lab-sheet-3>
```

Task 3: Indexed File Allocation

Objective: Write a Python program to simulate indexed file allocation strategy

```
total_blocks = int(input("Enter total number of blocks: "))
block_status = [0] * total_blocks

n = int(input("Enter number of files: "))

for i in range(n):
    index = int(input(f"Enter index block for file {i+1}: "))

    if block_status[index] == 1:
        print("Index block already allocated.")
        continue

    count = int(input("Enter number of data blocks: "))    data_blocks
    = list(map(int, input("Enter block numbers: ").split()))

    # Validation: correct count & ensure blocks are free    if len(data_blocks) !=
count or any(block_status[blk] == 1 for blk in data_blocks):
        print("Block(s) already allocated or invalid input.")
        continue

    block_status[index] = 1

    # Allocate data blocks
    for blk in data_blocks:
        block_status[blk] = 1

    print(f"File {i+1} allocated with index block {index} -> {data_blocks}")
```

Results:

```
D:\Lab-sheet-3>"C:/Program Files/Python312/python.exe" d:/Lab-sheet-3/task3.py
Enter total number of blocks: 10
Enter number of files: 2
Enter index block for file 1: 3
Enter number of data blocks: 2
Enter block numbers: 5 6
File 1 allocated with index block 3 -> [5, 6]
Enter index block for file 2: 2
Enter number of data blocks: 3
Enter block numbers: 8 0 9
File 2 allocated with index block 2 -> [8, 0, 9]
D:\Lab-sheet-3>
```

Task 4: Contiguous Memory Allocation

Objective: Simulate Worst-fit, Best-fit, and First-fit memory allocation strategies.

```
def allocate_memory(strategy):
    partitions = list(map(int, input("Enter partition sizes: ").split()))
    processes = list(map(int, input("Enter process sizes: ").split()))

    allocation = [-1] * len(processes)

    for i, psize in enumerate(processes):
        idx = -1

        if strategy == "first":
            # First Fit
            for j, part in enumerate(partitions):
                if part
                >= psize:
                    idx = j
            break

        elif strategy == "best":
            # Best Fit
            best_fit =
            float("inf")
            for j, part in enumerate(partitions):
                if part >=
                psize and part < best_fit:
                    best_fit = part
                    idx = j

        elif strategy == "worst":
            # Worst Fit
            worst_fit = -1
```

```

        for j, part in enumerate(partitions):
if part >= psize and part > worst_fit:
            worst_fit = part
            idx = j

        # Allocate if partition found
if idx != -1:
            allocation[i] = idx
            partitions[idx] -= psize

        # Output    for i, a in
enumerate(allocation):    if a !=
-1:
            print(f"Process {i+1} allocated in Partition {a+1}")
else:
            print(f"Process {i+1} cannot be allocated")

# Calling all three strategies print("\n--
- FIRST FIT ---")
allocate_memory("first")

print("\n--- BEST FIT ---")
allocate_memory("best")

print("\n--- WORST FIT ---") allocate_memory("worst")

```

Results:

```

D:\Lab-sheet-3>"C:/Program Files/Python312/python.exe" d:/Lab-sheet-3/task4.py

--- FIRST FIT ---
Enter partition sizes: 100 500 200 300
Enter process sizes: 47 58 35 28
Process 1 allocated in Partition 1
Process 2 allocated in Partition 2
Process 3 allocated in Partition 1
Process 4 allocated in Partition 2

--- BEST FIT ---
Enter partition sizes: 100 500 200 300
Enter process sizes: 47 58 35 28
Process 1 allocated in Partition 1
Process 2 allocated in Partition 3
Process 3 allocated in Partition 1
Process 4 allocated in Partition 3

--- WORST FIT ---
Enter partition sizes: 100 500 200 300
Enter process sizes: 47 58 35 28
Process 1 allocated in Partition 2
Process 2 allocated in Partition 2
Process 3 allocated in Partition 2
Process 4 allocated in Partition 2

```

Task 5: MFT & MVT Memory Management

Objective: Implement MFT (fixed partitions) and MVT (variable partitions) strategies in Python.

```
def MFT():
    mem_size = int(input("Enter total memory size: "))
    part_size = int(input("Enter partition size: "))
    n = int(input("Enter number of processes: "))

    partitions = mem_size // part_size
    print(f'Memory divided into {partitions} partitions')

    for i in range(n):
        psize = int(input(f'Enter size of Process {i+1}: '))
        if psize <= part_size:
            print(f'Process {i+1} allocated.')
        else:
            print(f'Process {i+1} too large for fixed partition.')

def MVT():
    mem_size = int(input("Enter total memory size: "))
    n = int(input("Enter number of processes: "))

    for i in range(n):
        psize = int(input(f'Enter size of Process {i+1}: '))
        if psize <= mem_size:
            print(f'Process {i+1} allocated.')
            mem_size -= psize
        else:
            print(f'Process {i+1} cannot be allocated. Not enough memory.')

print("MFT Simulation:")
MFT()

print("\nMVT Simulation:")
MVT()
```

Results:

```
D:\Lab-sheet-3>"C:/Program Files/Python312/python.exe" d:/Lab-sheet-3/task5.py
```

```
MFT Simulation:
```

```
Enter total memory size: 1000
```

```
Enter partition size: 200
```

```
Enter number of processes: 4
```

```
Memory divided into 5 partitions
```

```
Enter size of Process 1: 120
```

```
Process 1 allocated.
```

```
Enter size of Process 2: 300
```

```
Process 2 too large for fixed partition.
```

```
Enter size of Process 3: 200
```

```
Process 3 allocated.
```

```
Enter size of Process 4: 50
```

```
Process 4 allocated.
```

```
MVT Simulation:
```

```
Enter total memory size: 1000
```

```
Enter number of processes: 4
```

```
Enter size of Process 1: 200
```

```
Process 1 allocated.
```

```
Enter size of Process 2: 120
```

```
Process 2 allocated.
```

```
Enter size of Process 3: 300
```

```
Process 3 allocated.
```

```
Enter size of Process 4: 50
```

```
Process 4 allocated.
```


LAB ASSIGNMENT 4

Problem Title:

System Calls, VM Detection, and File System Operations using Python

Tools/Technology Used:

- System Calls
- VM Detection
- File System Operations using Python

Problem Statement:

Operating systems expose low-level interfaces like system calls to allow interaction between user programs and the OS kernel. This lab simulates system-level OS tasks such as process creation (using fork and exec), file and memory operations, VM detection, and CPU scheduling. Learners will develop shell, C, and Python scripts to model batch execution, inter-process communication, and basic file system behaviors.

Learning Objectives:

1. Execute Python scripts in a batch processing environment
2. Simulate system startup and process termination logging
3. Demonstrate use of system calls (fork(), exec(), wait(), pipe())
4. Detecting virtual machine environments programmatically
5. Simulate basic file system operations using Python

Sub Tasks :

Task 1: Batch Processing Simulation (Python)

Objective: Write a Python script to execute multiple .py files sequentially, mimicking batch processing.

import subprocess
scripts = ['script1.py', 'script2.py', 'script3.py']
for script in scripts: print(f"Executing {script}...") subprocess.call(['python3', script])

Task 2: System Startup and Logging

Objective: Simulate system startup using Python by creating multiple processes and logging their start and end into a log file.

```
import multiprocessing
import logging
import time

logging.basicConfig( filename='system_log.txt',
                    level=logging.INFO, format='%(asctime)s -
%(processName)s - %(message)s'
                    )

def process_task(name):
    logging.info(f'{name} started')
    time.sleep(2)
    logging.info(f'{name} terminated')

if __name__ == '__main__':
    print("System Booting...")

    p1 = multiprocessing.Process(target=process_task, args=("Process-1",))
    p2 = multiprocessing.Process(target=process_task, args=("Process-2",))

    p1.start()
    p2.start()

    p1.join()

    p2.join()

    print("System Shutdown.")
```

Task 3: System Calls and IPC (Python - fork, exec, pipe)

Objective: Use system calls (fork(), exec(), wait()) and implement basic Inter-Process Communication using pipes in C or Python.

```
import os

r, w = os.pipe()
pid = os.fork()

if pid > 0:
    # Parent process
    os.close(r)
    os.write(w,
b"Hello from parent")
    os.close(w)
    os.wait()

else:
    # Child process
    os.close(w)
    message = os.read(r, 1024)
    print("Child
received:", message.decode())
    os.close(r)
```

Task 4: VM Detection and Shell Interaction

Objective: Create a shell script to print system details and a Python script to detect if the system is running inside a virtual machine.

#!/bin/bash
echo "Kernel Version:" `uname -r`
echo "User:"
whoami
echo "Hardware Info:" `lscpu   grep "Virtualization"`

Task 5: CPU Scheduling Algorithms

Objective: Implement FCFS, SJF, Round Robin, and Priority Scheduling algorithms in Python to calculate WT and TAT.

```
# Priority Scheduling and Round Robin
processes = []

n = int(input("Enter number of processes: "))

# Input burst time and priority
for i in range(n):
    bt = int(input(f"Enter Burst Time for P{i+1}: "))
    pr = int(input(f"Enter Priority (lower number = higher priority) for P{i+1}: "))
    processes.append([i+1, bt, pr])

# Sort by priority (ascending)
processes.sort(key=lambda x: x[2])

wt = 0
total_wt = 0
total_tat = 0

print("\nPriority Scheduling:")
print("PID\tBT\tPriority\tWT\tTAT")

for pid, bt, pr in processes:
    tat = wt + bt # turnaround time
    print(f'{pid}\t{bt}\t{pr}\t{wt}\t{tat}')
```

```

    total_wt += wt    total_tat += tat    wt += bt #
update waiting time for next process

print(f"\nAverage Waiting Time: {total_wt / n:.2f}") print(f"Average
Turnaround Time: {total_tat / n:.2f}")

# FCFS
processes = []

n = int(input("Enter number of processes: "))

for i in range(n):
    bt = int(input(f"Enter Burst Time for P{i+1}: "))
    processes.append([i+1, bt]) # [PID, BT]

wt = 0 total_wt
= 0 total_tat =
0

print("\nFCFS Scheduling:")
print("PID\tBT\tWT\tTAT")

for pid, bt in processes:
    tat = wt + bt
    print(f"{pid}\t{bt}\t{wt}\t{tat}")
    total_wt += wt    total_tat += tat
    wt += bt

print(f"\nAverage Waiting Time: {total_wt/n:.2f}") print(f"Average
Turnaround Time: {total_tat/n:.2f}")

# SJF
processes = []

n = int(input("\nEnter number of processes for SJF: "))

for i in range(n):
    bt = int(input(f"Enter Burst Time for P{i+1}: "))
    processes.append([i+1, bt])

```

```
# Sort by burst time (ascending)
processes.sort(key=lambda x: x[1])

wt = 0 total_wt
= 0 total_tat =
0

print("\nSJF Scheduling:")
print("PID\tBT\tWT\tTAT")

for pid, bt in processes:
    tat = wt + bt
    print(f"{pid}\t{bt}\t{wt}\t{tat}")
    total_wt += wt    total_tat += tat
    wt += bt

print(f"\nAverage Waiting Time: {total_wt/n:.2f}") print(f"Average
Turnaround Time: {total_tat/n:.2f}")
```